

BriefDaily

Personalized News Digest and Bookmark Application

**Personalized News
Digest Platform**

Complete Technical Project Documentation

Flutter and Dart Application

GitHub Repository: github.com/vijayKota2776/BriefDaily

Figma Design: [BriefDaily Figma Design](#)

Abstract

BriefDaily is a personalized news digest and bookmarking application developed using Flutter and Dart. The application addresses the problem of information overload by allowing users to select topics of interest and receive a focused news digest based on those preferences.

The application combines reactive state management, remote news retrieval, authentication, local persistence, bookmarking, article reading, text-to-speech functionality, and responsive Material 3 user interface design.

The main application flow consists of authentication, onboarding, topic selection, personalized news retrieval, article reading, bookmarking, and profile management. User preferences are managed using Riverpod, while persistent local data is stored using Hive. Firebase Authentication is used for user authentication and NewsAPI provides real-world news content.

The project also includes a Figma design system covering desktop and mobile application screens.

This document describes the complete architecture and development process of BriefDaily, including the major technical challenges, solutions, limitations, testing strategy, and potential future improvements.

Contents

1	Project Overview	10
1.1	Introduction	10
1.2	Problem Statement	10
1.3	Project Objectives	11
1.4	Target Users	11
2	Functional Requirements	12
2.1	Authentication	12
2.2	Topic Selection	12
2.3	News Digest	13
2.4	Article Reading	13
2.5	Bookmarks	14
2.6	Bookmark Removal	14
2.7	Responsive Navigation	14
3	Technology Stack	15
3.1	Technology Overview	15
3.2	Flutter	15
3.3	Dart	16
3.4	Riverpod	16
3.5	Hive	17
3.6	Firebase Authentication	17

3.7	NewsAPI	17
4	System Architecture	18
4.1	Architectural Philosophy	18
4.2	High-Level Architecture	18
4.3	Feature-First Structure	19
4.4	Presentation Layer	19
4.5	State Layer	19
4.6	Data Layer	20
5	Application Data Flow	21
5.1	Authentication Flow	21
5.2	Topic Selection Flow	21
5.3	News Retrieval Flow	21
5.4	Bookmark Flow	22
6	Data Models	23
6.1	Article Model	23
6.2	Article Identity	23
6.3	Publication Date	23
7	News Retrieval and Personalization	25
7.1	Personalization Model	25
7.2	Topic Filtering	25
7.3	Newest-First Ordering	26
7.4	API Data Transformation	26
8	Authentication Architecture	27
8.1	Firebase Authentication	27
8.2	Authentication State	27
8.3	Logout	27

9	Offline Persistence	28
9.1	Purpose	28
9.2	Bookmark Serialization	28
9.3	Why Complete Article Data Is Stored	28
9.4	Persistence Lifecycle	28
10	User Interface and UX Design	30
10.1	Design Philosophy	30
10.2	Material 3	30
10.3	Inter Typography	31
10.4	Article Cards	31
10.5	Bookmark Interaction	32
11	Figma Design	33
11.1	Design Tool	33
11.2	Desktop Screens	33
11.3	Mobile Screens	33
11.4	Visual Language	34
11.5	Figma Resource	34
12	Major Development Challenges	35
12.1	Challenge 1: Limited Disk Space	35
12.2	Challenge 2: Figma Pixel Alignment	35
12.3	Challenge 3: Live API Filtering	36
12.4	Challenge 4: Authentication State Duplication	36
12.5	Challenge 5: Bookmark Persistence	36
13	State Management	37
13.1	Riverpod Architecture	37
13.2	Reactive Topic Selection	37

13.3 Bookmark Provider	38
14 Project Structure	39
14.1 Logical Structure	39
14.2 Separation of Concerns	40
15 Testing Strategy	41
15.1 Testing Objectives	41
15.2 Current Unit Tests	41
15.3 Model Testing	41
15.4 Digest Sorting Tests	42
15.5 Recommended Additional Testing	42
16 Security Considerations	43
16.1 Authentication Security	43
16.2 API Key Management	43
16.3 Local Storage	43
16.4 Network Communication	43
17 Performance Considerations	45
17.1 Network Requests	45
17.2 Image Loading	45
17.3 List Rendering	45
17.4 State Updates	45
18 Complete User Experience Flow	46
18.1 Application Launch	46
18.2 Digest Flow	46
18.3 Reading Flow	47
18.4 Bookmark Flow	47

19 Coursework Requirement Mapping	48
20 Development Retrospective	50
20.1 Initial Development	50
20.2 Data Integration	50
20.3 Personalization	50
20.4 Persistence	50
20.5 Authentication	51
20.6 Visual Refinement	51
21 Known Limitations	52
21.1 External API Dependency	52
21.2 API Configuration	52
21.3 Figma Prototype Interactions	52
21.4 Test Coverage	52
21.5 Authentication State	52
22 Future Enhancements	53
22.1 Advanced Personalization	53
22.2 AI-Assisted Summaries	53
22.3 Offline News Cache	53
22.4 Push Notifications	54
22.5 Advanced Analytics	54
22.6 Improved Testing	54
23 Lessons Learned	55
23.1 State Management	55
23.2 Persistence Design	55
23.3 API Integration	55
23.4 Design and Development	56

23.5 Security	56
24 Final Technical Assessment	57
25 Conclusion	58
A Useful Development Commands	59
A.1 Install Dependencies	59
A.2 Run Application	59
A.3 Clean Build	59
A.4 Static Analysis	59
A.5 Run Tests	59
B Example Riverpod Flow	60
C Example Article JSON	61
D Project Summary	62
E Project Resources	63
E.1 GitHub Repository	63
E.2 Figma Design	63
E.3 Quick Access	64
E.4 Resource Usage	64

List of Tables

3.1	BriefDaily Technology Stack	15
19.1	Coursework Requirement Mapping	48
D.1	Project Summary	62
E.1	BriefDaily Project Resources	64

List of Figures

4.1 BriefDaily High-Level Architecture	18
--	----

1 Project Overview

1.1 Introduction

BriefDaily is a personalized news digest application designed to help users consume relevant news without having to manually search through large numbers of articles.

The central idea is simple: users select the topics they are interested in and the application generates a digest containing news articles related to those interests.

The project was developed as a Flutter application with a focus on:

- Clean Material 3 interface design.
- Reactive application state.
- Personalized content filtering.
- Persistent local bookmarks.
- Secure authentication.
- Responsive layouts.
- Real-world API integration.
- Offline access to saved articles.

1.2 Problem Statement

Modern users are exposed to a large amount of news every day. Searching through multiple news sources can become time-consuming and can make it difficult to focus on topics that are personally relevant.

BriefDaily addresses this problem by providing a personalized digest based on user-selected topics.

The application reduces the number of interactions required to discover relevant content while still allowing the user to explore complete articles when required.

1.3 Project Objectives

The primary objectives of the project are:

1. Allow users to authenticate securely.
2. Allow users to select topics of interest.
3. Retrieve current news articles from a remote API.
4. Filter news according to selected topics.
5. Display articles in a clean editorial interface.
6. Sort articles by publication time.
7. Allow users to bookmark articles.
8. Persist bookmarks locally.
9. Allow users to remove saved articles.
10. Provide an article reading experience.
11. Support text-to-speech functionality.
12. Provide responsive layouts for different screen sizes.

1.4 Target Users

The application is intended for users who:

- Want a short personalized news digest.
- Follow specific topics.
- Prefer a clean reading experience.
- Want to save articles for later.
- Need access to saved articles without continuously retrieving them from the API.

2 Functional Requirements

2.1 Authentication

The application provides an authentication flow using Firebase Authentication.

The authentication system supports the application lifecycle:

1. User opens the application.
2. User signs in or creates an account.
3. Firebase validates the credentials.
4. Authenticated users enter the application.
5. The application loads the user's preferences.

2.2 Topic Selection

Users can select topics that determine the content shown in the digest.

Examples of topics include:

- Technology
- Business
- Science
- Sports
- Entertainment
- Health

- Politics
- World

Topic selection is represented through interactive Material components such as FilterChip widgets.

2.3 News Digest

The News Digest is the primary content screen.

Each article card is intended to provide:

- Headline.
- Source.
- Topic tag.
- Publication time.
- Article image when available.
- Reading time.
- Bookmark control.

The digest is ordered using publication timestamps so that recently published content appears first.

2.4 Article Reading

Selecting an article opens an article reading interface.

The reading interface provides:

- Article title.
- Source information.
- Publication information.
- Article content or summary.

- Bookmark functionality.
- Text-to-speech functionality where supported.

2.5 Bookmarks

Users can save articles from the digest or reading interface.

Bookmarks are stored locally using Hive.

The saved object contains the complete article information rather than only an article identifier. This allows bookmarked content to remain available when the device is offline.

2.6 Bookmark Removal

Users can remove individual bookmarks.

The interface supports swipe-to-delete behavior and provides an undo interaction after deletion.

2.7 Responsive Navigation

The application adapts navigation according to screen width.

On smaller screens, a bottom navigation interface is used.

On larger screens, a navigation rail can be used to provide more efficient navigation.

3 Technology Stack

3.1 Technology Overview

The application uses the following technologies.

Table 3.1: BriefDaily Technology Stack

Technology	Purpose	Role in Project
Flutter	Application framework	Cross-platform UI development
Dart	Programming language	Application logic
Riverpod	State management	Reactive application state
Hive	Local database	Persistent bookmarks and preferences
Firebase Auth	Authentication	User authentication
NewsAPI	News provider	Remote news retrieval
HTTP	Networking	API communication
Material 3	UI system	Application visual language
Google Fonts	Typography	Inter typography
Cached Network Image	Image loading	Efficient article image rendering
Flutter TTS	Text-to-speech	Article audio playback
Figma	UI/UX design	Interface design and prototyping

3.2 Flutter

Flutter provides the main cross-platform application framework.

The project uses Flutter widgets to create:

- Screens.
- Cards.
- Navigation.
- Buttons.
- Topic selection controls.
- Article layouts.
- Responsive interfaces.

3.3 Dart

Dart is used for all application logic.

It provides:

- Strong typing.
- Object-oriented programming.
- Asynchronous programming.
- Futures and streams.
- Null safety.

3.4 Riverpod

Riverpod manages application state and allows widgets to react to changes in user preferences, articles, and bookmarks.

The general state flow is:

Preferences → Riverpod Provider → News Service → Digest State → UI

When selected topics change, dependent providers can react and refresh the digest.

3.5 Hive

Hive provides local persistence.

It is used for data that should remain available after the application is closed.

Examples include:

- User preferences.
- Selected topics.
- Bookmark data.

3.6 Firebase Authentication

Firebase Authentication handles user identity and authentication operations.

3.7 NewsAPI

NewsAPI is used as the external source of news articles.

The application requests content based on the selected topic preferences and converts the returned information into application models.

4 System Architecture

4.1 Architectural Philosophy

BriefDaily follows a feature-first organization strategy.

Instead of creating one large directory for every model, widget, repository, and screen, functionality is grouped according to what the application feature does.

This improves maintainability because related functionality can be located together.

4.2 High-Level Architecture

The application can be represented using the following conceptual architecture:

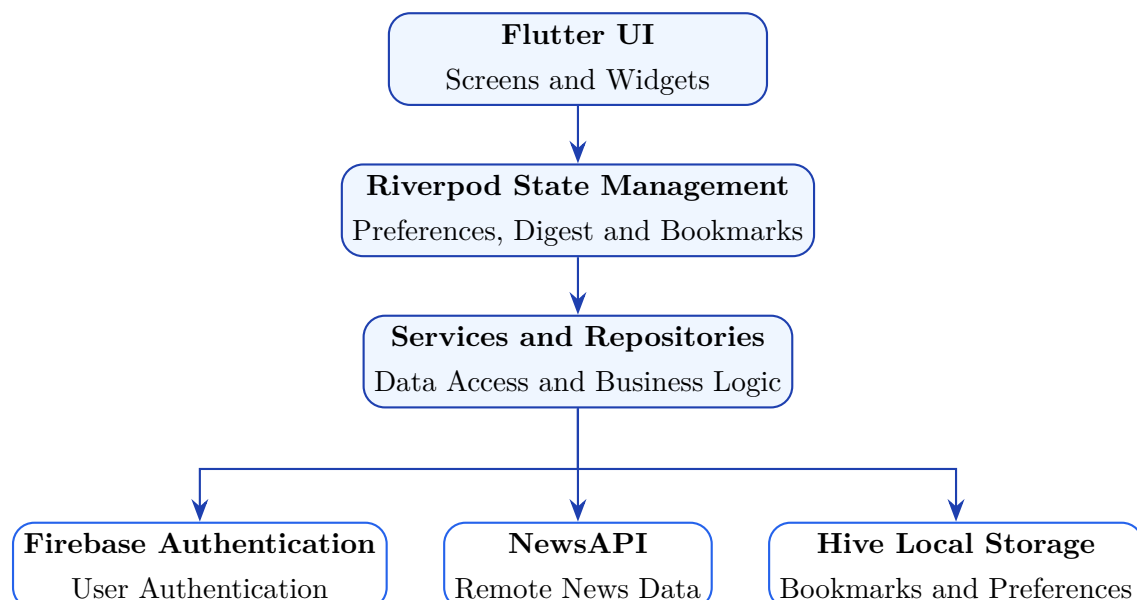


Figure 4.1: BriefDaily High-Level Architecture

The architecture separates the presentation, state-management, application, external-service, and local-persistence responsibilities.

4.3 Feature-First Structure

The conceptual project structure is:

```
lib/  
|-- app/  
|-- data/  
|   '-- repositories/  
|-- features/  
|   |-- auth/  
|   |-- digest/  
|   |   |-- article/  
|   |   '-- bookmarks/  
|   |-- profile/  
|   '-- onboarding/  
|-- models/  
|-- providers/  
|-- services/  
'-- main.dart
```

The structure separates application concerns while keeping feature implementation discoverable.

4.4 Presentation Layer

The presentation layer contains:

- Screens.
- Widgets.
- Navigation.
- User interactions.

Examples include the login screen, topic selection screen, digest screen, article screen, bookmarks screen, and profile screen.

4.5 State Layer

The state layer contains Riverpod providers and controllers.

This layer manages:

- User preferences.
- Authentication-related application state.
- News digest state.
- Bookmark state.

4.6 Data Layer

The data layer is responsible for retrieving and persisting data.

It includes:

- News API communication.
- Bookmark persistence.
- Local preferences.
- Data serialization.

5 Application Data Flow

5.1 Authentication Flow

The authentication flow can be represented as:

1. Application starts.
2. Firebase authentication state is checked.
3. If the user is authenticated, the application loads preferences.
4. If authentication is unavailable, the login interface is shown.
5. Successful authentication leads to the application shell.

5.2 Topic Selection Flow

The topic selection process follows:

User Selection → Preferences Provider → Persistent Storage → Digest Provider →
News Retrieval → Updated Digest

The important characteristic of this flow is that the selected topics are reactive state rather than static configuration.

5.3 News Retrieval Flow

The news retrieval process consists of:

1. Read the selected topics.

2. Build the news request.
3. Send the request to NewsAPI.
4. Parse the response.
5. Convert results to Article models.
6. Remove invalid or unusable results.
7. Sort by publication timestamp.
8. Expose the resulting list through application state.

5.4 Bookmark Flow

The bookmark flow is:

Article → Bookmark Action → Bookmark Provider → Article Serialization → Hive

When the application is reopened, saved article data can be loaded from Hive.

6 Data Models

6.1 Article Model

The central application model is the Article model.

The model contains information such as:

```
Article {  
    id  
    title  
    summary  
    content  
    source  
    topic  
    author  
    readingTime  
    imageUrl  
    url  
    publishedAt  
}
```

6.2 Article Identity

Each article should have a stable identifier.

This identifier can be used to determine whether an article is already bookmarked.

6.3 Publication Date

The publishedAt property represents the publication timestamp.

It is also used for digest ordering.

The conceptual sorting operation is:

```
articles.sort(  
    (a, b) => b.publishedAt.compareTo(a.publishedAt)  
);
```

This ensures that newer articles appear before older articles.

7 News Retrieval and Personalization

7.1 Personalization Model

BriefDaily uses topic preferences as the primary personalization mechanism.

For example, if a user selects:

- Technology
- Business
- Science

the application should retrieve and display content related to these selected interests.

7.2 Topic Filtering

Topic changes should immediately affect the digest.

The intended reactive relationship is:

Selected Topics

|

v

Preferences Provider

|

v

Digest Provider

|

v

News Service

|

v

```
Sorted Article List
```

```
|
```

```
v
```

```
Digest UI
```

7.3 Newest-First Ordering

The digest is explicitly sorted according to publication date.

This is important because external APIs may not always guarantee the exact ordering expected by the application.

The application therefore performs its own final ordering.

7.4 API Data Transformation

External API responses are converted into the internal Article model.

This provides separation between the external API representation and the rest of the application.

Conceptually:

```
NewsAPI Response
```

```
|
```

```
v
```

```
JSON Parsing
```

```
|
```

```
v
```

```
Article.fromJson()
```

```
|
```

```
v
```

```
Article Model
```

```
|
```

```
v
```

```
Riverpod State
```

```
|
```

```
v
```

```
UI
```

8 Authentication Architecture

8.1 Firebase Authentication

Firebase Authentication provides the external identity service.

The application does not need to implement password cryptography or authentication infrastructure itself.

Firebase manages the authentication operation while the application reacts to authentication state.

8.2 Authentication State

Authentication state should have a single authoritative source.

Firebase Authentication is the authoritative source for whether a user is authenticated.

Local application storage can retain application preferences, but it should not independently determine whether a Firebase user is authenticated.

8.3 Logout

The conceptual logout operation is:

```
await FirebaseAuth.instance.signOut();
```

Application-level state should then react to the authentication change.

This avoids conflicting authentication states.

9 Offline Persistence

9.1 Purpose

Offline persistence is primarily used for bookmarks and preferences.

A user should not lose saved articles merely because the application is closed or temporarily disconnected from the internet.

9.2 Bookmark Serialization

The application serializes the complete Article model.

A conceptual serialization process is:

```
final encodedArticle =  
    json.encode(article.toJson());
```

The encoded object can then be stored in Hive.

9.3 Why Complete Article Data Is Stored

Storing only an article ID would require the application to retrieve the article again from the remote service.

By storing the complete article representation, BriefDaily can provide access to saved article information even when the network is unavailable.

9.4 Persistence Lifecycle

The lifecycle is:

1. User bookmarks an article.
2. Article is converted into JSON.
3. JSON is written to Hive.
4. Bookmark provider updates application state.
5. UI reflects the new bookmark.
6. On application restart, Hive is read.
7. Stored articles are reconstructed.

10 User Interface and UX Design

10.1 Design Philosophy

The interface follows a modern editorial visual style.

The design focuses on:

- Strong headline hierarchy.
- Compact information presentation.
- Clear topic indicators.
- Consistent spacing.
- Rounded interface components.
- Material 3 interaction patterns.
- Dark visual presentation.

10.2 Material 3

The application enables Material 3 through the Flutter theme system.

The design uses Material components including:

- `NavigationBar`.
- `NavigationRail`.
- `FilterChip`.
- `Cards`.

- Buttons.
- Icons.

10.3 Inter Typography

Inter is used as the primary visual typeface.

The typography hierarchy separates:

- Page titles.
- Article headlines.
- Metadata.
- Supporting descriptions.
- Navigation labels.

This produces a consistent visual hierarchy.

10.4 Article Cards

Article cards are the primary reusable content component.

A card can contain:

- Article image.
- Headline.
- Source.
- Topic.
- Publication time.
- Reading time.
- Bookmark action.

10.5 Bookmark Interaction

Bookmarks use a visual icon-based interaction.

A saved article is visually distinguished from an unsaved article.

The interaction should provide immediate feedback to the user.

11 Figma Design

11.1 Design Tool

Figma was used to develop the visual design and application flow.

The design includes desktop and mobile screen layouts.

11.2 Desktop Screens

The desktop design contains the following screens:

1. Demo 01 – Login
2. Demo 02 – Welcome
3. Demo 03 – Choose Topics
4. Demo 04 – Personalizing
5. Demo 05 – Your Brief
6. Demo 06 – Article Reading
7. Demo 07 – Saved Stories
8. Demo 08 – Profile and Settings

11.3 Mobile Screens

The mobile design contains:

1. Mobile 01 – Login

2. Mobile 02 – Welcome
3. Mobile 03 – Choose Topics
4. Mobile 04 – Personalizing
5. Mobile 05 – Your Brief
6. Mobile 06 – Article Reading
7. Mobile 07 – Saved Stories
8. Mobile 08 – Profile and Settings

11.4 Visual Language

The visual design uses:

- Dark surfaces.
- Blue accent.
- Inter typography.
- Rounded cards.
- Topic chips.
- Editorial article layouts.
- Bottom navigation on mobile.
- Navigation rail on larger layouts.

11.5 Figma Resource

The complete editable Figma design can be accessed through the following link:

[Open the BriefDaily Figma Design](#)

12 Major Development Challenges

12.1 Challenge 1: Limited Disk Space

During development, the project encountered a disk-space problem associated with Flutter dependencies and build artifacts.

The reported environment had less than approximately 480 MB of available storage.

This resulted in an error similar to:

```
Errno 28  
No space left on device
```

The issue was addressed by cleaning unnecessary Flutter build artifacts, removing stale cache data, and freeing storage.

Typical cleanup commands included:

```
flutter clean
```

Additional stale cache and build files were removed when necessary.

12.2 Challenge 2: Figma Pixel Alignment

The initial implementation did not exactly match the intended Figma design.

The main differences involved:

- Typography.
- Card dimensions.
- Topic tiles.
- Spacing.

- Bookmark icon treatment.
- Color values.

The interface was refined using Inter typography, compact topic tiles, consistent spacing, and the intended blue accent.

12.3 Challenge 3: Live API Filtering

An early news retrieval implementation did not correctly account for all selected topics.

The implementation was subsequently improved so that selected topics could participate in the news retrieval process and the final article collection was explicitly sorted by publication time.

12.4 Challenge 4: Authentication State Duplication

A major architectural issue involved maintaining authentication state both locally and through Firebase.

This could result in two different representations of authentication state.

The improved approach is to treat Firebase Authentication as the authoritative authentication source and use local storage for application preferences rather than independent authentication truth.

12.5 Challenge 5: Bookmark Persistence

An early bookmark implementation stored only article identifiers.

This created a dependency on retrieving article content again.

The implementation was improved so that complete Article data is serialized and stored in Hive.

13 State Management

13.1 Riverpod Architecture

Riverpod is used to provide reactive state management.

The primary state categories include:

- Preferences.
- Digest data.
- Bookmarks.
- Authentication-related state.

13.2 Reactive Topic Selection

The desired relationship is:

```
User changes topic
      |
      v
Preferences Provider changes
      |
      v
Dependent digest state refreshes
      |
      v
New articles are loaded
      |
      v
Digest UI rebuilds
```

This prevents the user from having to manually refresh the screen after changing interests.

13.3 Bookmark Provider

The bookmark provider acts as the reactive layer between the UI and local persistence.

It handles:

- Adding bookmarks.
- Removing bookmarks.
- Loading persisted bookmarks.
- Updating UI state.

14 Project Structure

14.1 Logical Structure

A simplified project structure is:

```
lib/  
|-- app/  
|   |-- theme/  
|   '-- routing/  
|  
|-- data/  
|   '-- repositories/  
|  
|-- features/  
|   |-- auth/  
|   |-- digest/  
|   |   |-- screens/  
|   |   '-- widgets/  
|   |-- onboarding/  
|   |-- profile/  
|   '-- bookmarks/  
|  
|-- models/  
|  
|-- providers/  
|  
|-- services/  
|  
'-- main.dart
```

14.2 Separation of Concerns

The architecture attempts to separate:

1. Presentation.
2. State management.
3. Data access.
4. External services.
5. Models.

This makes the application easier to maintain and extend.

15 Testing Strategy

15.1 Testing Objectives

Testing is intended to verify:

- Data model behavior.
- JSON serialization.
- Preference state.
- Digest sorting.
- Bookmark behavior.
- User interface behavior.

15.2 Current Unit Tests

The project contains tests covering areas such as:

```
test/  
|-- article_test.dart  
|-- digest_provider_test.dart  
'-- models/  
    |-- article_test.dart  
    '-- user_preferences_test.dart
```

15.3 Model Testing

Article model tests verify construction and serialization behavior.

Conceptually:

```
final article = Article(...);

final jsonData = article.toJson();

final restored =
    Article.fromJson(jsonData);
```

The restored object should retain the relevant article properties.

15.4 Digest Sorting Tests

Digest logic should verify that publication timestamps produce a newest-first list.

For example:

```
final sortedArticles = List<Article>.from(articles)
    ..sort(
        (a, b) =>
            b.publishedAt.compareTo(a.publishedAt),
    );
```

15.5 Recommended Additional Testing

Additional tests should cover:

- Complete digest provider behavior.
- Topic filtering.
- Bookmark UI interactions.
- Authentication transitions.
- Navigation.
- Responsive layouts.
- Error states.
- Empty states.
- Network failures.

16 Security Considerations

16.1 Authentication Security

Firebase Authentication should remain responsible for authentication credentials and identity management.

The application should not implement its own password storage system.

16.2 API Key Management

External API keys should not be permanently embedded as plain-text secrets in production application source code.

A production implementation should use an appropriate configuration or backend proxy strategy.

If an API key has previously been committed to a public repository, it should be treated as exposed and rotated.

16.3 Local Storage

Hive is appropriate for application-level local persistence.

Sensitive credentials should not be stored in ordinary application storage.

16.4 Network Communication

Remote requests should use HTTPS endpoints.

API failures should be handled gracefully without exposing internal implementation de-

tails to the user.

17 Performance Considerations

17.1 Network Requests

Network requests should avoid unnecessary duplication.

Caching and local persistence can reduce repeated remote operations.

17.2 Image Loading

Article images can be loaded using cached network image functionality.

Caching reduces repeated downloads when the same image is displayed multiple times.

17.3 List Rendering

News articles are displayed using Flutter list-based widgets.

Lazy list rendering helps prevent the application from constructing every article widget simultaneously.

17.4 State Updates

Riverpod allows only relevant consumers to rebuild when state changes, reducing unnecessary UI work.

18 Complete User Experience Flow

18.1 Application Launch

The overall user journey can be described as:

1. Launch BriefDaily.
2. Check authentication state.
3. Sign in or create an account.
4. Complete onboarding.
5. Select preferred topics.
6. Personalization state is saved.
7. Open the personalized digest.

18.2 Digest Flow

After onboarding:

1. The selected topics are read.
2. News is requested.
3. Articles are converted into application models.
4. Articles are sorted by publication time.
5. The digest displays the resulting list.

18.3 Reading Flow

The reading flow is:

1. User selects an article.
2. Article reading screen opens.
3. User reads the content.
4. User can bookmark the article.
5. User can optionally use text-to-speech.

18.4 Bookmark Flow

The bookmark flow is:

1. User presses the bookmark icon.
2. Article data is serialized.
3. Article is saved in Hive.
4. Bookmark provider updates.
5. Saved Stories screen displays the article.

19 Coursework Requirement Mapping

Table 19.1: Coursework Requirement Mapping

Requirement	Status	Implementation
Topic Selection	Implemented	Topic selection using interactive chips and persisted preferences.
News Digest	Implemented	NewsAPI-backed digest displaying current article data.
Bookmarks	Implemented	Complete article data stored locally using Hive.
Remove Bookmarks	Implemented	Individual bookmark removal with swipe interaction and undo behavior.
Riverpod	Implemented	Reactive state management for preferences and application data.
Material 3	Implemented	Flutter Material 3 theme and components.
Editorial Typography	Implemented	Inter typography and structured content hierarchy.
Headline	Implemented	Article cards display headline information.
Source	Implemented	Article cards display source information.
Topic Tag	Required	Topic metadata should be visible on every article card.
Publish Time	Implemented	Publication timestamp is displayed using relative time formatting.

Newest First	Implemented	Articles are explicitly sorted using publishedAt descending.
Topic Filtering	Implemented	Selected topics influence digest content.
Persistent Storage	Implemented	Hive is used for local persistence.
Responsive UI	Implemented	Mobile navigation and larger-screen navigation rail layouts.
Figma Design	Implemented	Desktop and mobile application screens created in Figma.
Automated Testing	Partial	Basic unit tests exist; broader widget and integration coverage can be expanded.

20 Development Retrospective

20.1 Initial Development

The initial project focused on establishing the basic Flutter structure, screen navigation, and visual language.

Material 3 components were selected as the foundation for the interface.

20.2 Data Integration

The project was then connected to real news data through NewsAPI.

This changed the application from a static prototype into an application capable of displaying current external content.

20.3 Personalization

Topic selection was connected to Riverpod state.

This created the central personalized experience of the application.

20.4 Persistence

Hive was introduced to preserve local application state and bookmarks.

The bookmark implementation was subsequently improved to store complete article information.

20.5 Authentication

Firebase Authentication was integrated to provide a real authentication system.

The authentication architecture was then refined to reduce duplicated sources of truth.

20.6 Visual Refinement

The UI was refined against the Figma design.

Important refinements included:

- Typography.
- Card spacing.
- Topic tile sizing.
- Bookmark visual treatment.
- Color consistency.
- Responsive navigation.

21 Known Limitations

21.1 External API Dependency

The digest depends on the availability and behavior of the external news service.

If the service is unavailable, the application must handle the error without crashing.

21.2 API Configuration

API credentials require secure configuration before a production deployment.

Hard-coded secrets should not be included in a public production repository.

21.3 Figma Prototype Interactions

The Figma design contains the major application screens and editable layouts. Some prototype interactions may require additional wiring before the design can be considered a completely interactive prototype.

21.4 Test Coverage

The existing unit tests provide a foundation, but a larger application would benefit from additional widget and integration tests.

21.5 Authentication State

Firebase should remain the authoritative authentication source. Any local preference representation must not conflict with the Firebase authentication lifecycle.

22 Future Enhancements

22.1 Advanced Personalization

Future versions could personalize the digest using:

- Reading history.
- Bookmark history.
- Article engagement.
- Topic weighting.
- Explicit user feedback.

22.2 AI-Assisted Summaries

The reading experience could be expanded with AI-generated summaries.

Potential capabilities include:

- Short article summaries.
- Key points.
- Simplified explanations.
- Topic-specific context.

22.3 Offline News Cache

Future versions could cache recently viewed articles in addition to bookmarks.

This would improve the reading experience during temporary network outages.

22.4 Push Notifications

Users could receive notifications for important articles related to their selected topics.

22.5 Advanced Analytics

The application could provide personal reading analytics such as:

- Articles read.
- Topics followed.
- Reading time.
- Bookmark count.
- Reading streaks.

22.6 Improved Testing

A complete production release should include:

- Unit tests.
- Widget tests.
- Integration tests.
- Authentication tests.
- Repository tests.
- Network failure tests.

23 Lessons Learned

23.1 State Management

One of the main lessons from the project was the importance of having clear ownership of application state.

Reactive state management makes UI synchronization easier, but only when each state has a clearly defined source of truth.

23.2 Persistence Design

Local persistence should be designed around actual application requirements.

For bookmarks, storing the complete article proved more useful than storing only identifiers because it supports offline access.

23.3 API Integration

External APIs should not be treated as perfectly structured sources.

Application code should:

- Validate responses.
- Handle missing fields.
- Handle invalid dates.
- Handle network failures.
- Explicitly sort application data.

23.4 Design and Development

Figma and Flutter should be treated as connected parts of the same design process.

Small differences in typography, spacing, and component sizing can significantly affect the perceived quality of an application.

23.5 Security

Development shortcuts involving credentials can create significant risks.

External keys and secrets should be separated from source code before a production release.

24 Final Technical Assessment

BriefDaily successfully demonstrates the architecture and implementation of a personalized news digest application using Flutter.

The project combines several important software engineering concepts:

- Cross-platform UI development.
- Reactive state management.
- REST API integration.
- Authentication.
- Local persistence.
- Responsive design.
- Component-based UI architecture.
- Automated testing.
- UX prototyping.

The application provides the major functionality required for the coursework, including topic selection, personalized news content, bookmarking, local persistence, responsive navigation, and Material 3 interface design.

The remaining areas requiring attention before describing the project as a production deployment include stronger automated test coverage, secure API key management, complete authentication-state verification, and final Figma prototype interaction wiring.

25 Conclusion

BriefDaily demonstrates how a relatively small Flutter application can combine real-world data, authentication, state management, persistence, and responsive user interface design into one coherent product.

The project began as a personalized news digest concept and evolved into a functional prototype containing authentication, onboarding, topic personalization, live news retrieval, article reading, bookmarks, offline persistence, responsive navigation, and text-to-speech functionality.

Riverpod provides the reactive state foundation, Firebase provides authentication, News-API provides external news content, and Hive provides local persistence.

The development process also demonstrated several practical engineering lessons, particularly around API integration, state ownership, persistence design, visual consistency, and security.

Overall, BriefDaily provides a strong technical foundation for further development toward a larger production-oriented news application.

A Useful Development Commands

A.1 Install Dependencies

```
flutter pub get
```

A.2 Run Application

```
flutter run
```

A.3 Clean Build

```
flutter clean  
flutter pub get
```

A.4 Static Analysis

```
flutter analyze
```

A.5 Run Tests

```
flutter test
```

B Example Riverpod Flow

The following conceptual example demonstrates how a preference can influence dependent application state.

```
final selectedTopicsProvider =
    StateNotifierProvider<PreferencesNotifier, List<String>>(
        (ref) => PreferencesNotifier(),
    );

final digestProvider =
    FutureProvider<List<Article>>((ref) async {

        final topics =
            ref.watch(selectedTopicsProvider);

        final service = NewsService();

        return service.fetchNews(topics);
    });
```

The important principle is that the digest observes the topic state. When the selected topics change, the dependent state can refresh.

C Example Article JSON

A serialized article can conceptually resemble:

```
{
  "id": "article-001",
  "title": "Example Technology Headline",
  "summary": "Short article summary.",
  "content": "Article content.",
  "source": "Example News",
  "topic": "Technology",
  "author": "Example Author",
  "readingTime": 4,
  "imageUrl": "https://example.com/image.jpg",
  "url": "https://example.com/article",
  "publishedAt": "2026-09-30T10:30:00Z"
}
```

This object can be serialized and stored locally for bookmark persistence.

D Project Summary

Table D.1: Project Summary

Property	Description
Project Name	BriefDaily
Application Type	Personalized News Digest and Bookmark App
Framework	Flutter
Language	Dart
State Management	Riverpod
Authentication	Firebase Authentication
Remote Data	NewsAPI
Local Storage	Hive
Design System	Material 3
Typography	Inter
Design Tool	Figma
Primary Feature	Personalized News Digest
Secondary Feature	Offline Bookmarks
Reading Feature	Article Reading and TTS
Responsive Support	Mobile and Larger Screens
GitHub Repository	BriefDaily GitHub Repository
Figma Design	BriefDaily Figma Design

E Project Resources

This section provides direct access to the primary project resources: the application source code and the UI/UX design files.

E.1 GitHub Repository

The complete BriefDaily application source code is maintained in the project's GitHub repository.

BriefDaily GitHub Repository

github.com/vijayKota2776/BriefDaily

The repository contains the Flutter application source code, application models, Riverpod providers, services, repositories, UI components, tests, configuration, and supporting project files.

E.2 Figma Design

The BriefDaily UI/UX design was created in Figma and contains the desktop and mobile layouts used as the visual reference for the application.

Open the BriefDaily Figma Design

[BriefDaily Figma Design](#)

The Figma file contains the major application screens covering:

- Login.
- Welcome and onboarding.
- Topic selection.
- Personalization.
- Personalized news digest.
- Article reading.
- Saved stories.
- Profile and settings.

E.3 Quick Access

Table E.1: BriefDaily Project Resources

Resource	Access Link
GitHub Source Code	Open GitHub Repository
Figma UI/UX Design	Open Figma Design

E.4 Resource Usage

The GitHub repository represents the implementation side of the BriefDaily project, while the Figma file represents the design and user-experience side.

Together, these resources provide access to:

1. Application source code.
2. Application architecture.
3. UI components.
4. State-management implementation.
5. Local persistence implementation.
6. Authentication integration.

7. News API integration.
8. Desktop UI designs.
9. Mobile UI designs.
10. User-flow and interface references.